

Lecture 3: OOP and File Handling for Backend Development

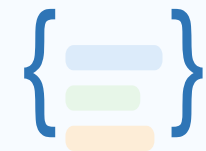
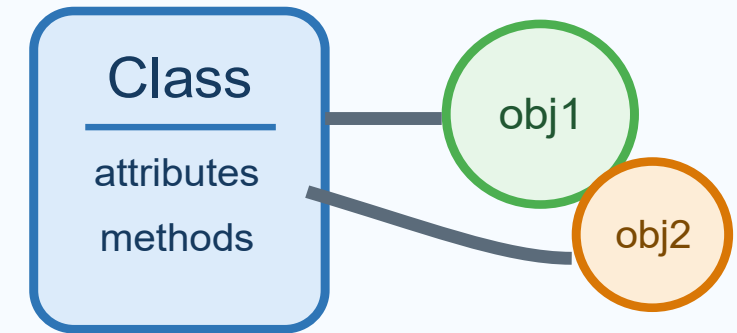
Deeper than Lecture 2 | For second-year FCAI students
| English teaching deck

Deep lecture deck with detailed explanation, live coding, class activities, and file-based mini tasks.

Teaching target

Students should leave this lecture able to create classes, instantiate objects, explain self, use inheritance, and read/write files confidently.

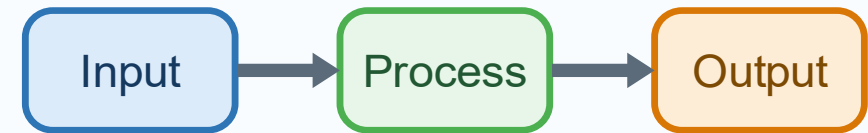
Dr. Mohamed Nour Eldien



Lecture Objectives

By the end of this lecture, students should be able to:

- Explain the purpose of object-oriented programming in Python.
- Differentiate between class, object, attribute, method, and constructor.
- Create classes and instantiate multiple objects from the same class.
- Understand inheritance, overriding, and simple design decisions.
- Read from and write to files using Python in a safe and organized way.
- Connect OOP and file handling to backend programming practice.



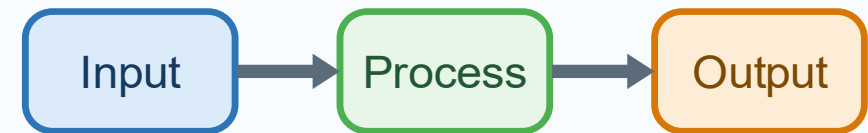
Suggested opening

Tell students that this lecture moves from “small scripts” to “organized programs”. That is the bridge to real backend projects.

Lecture Roadmap

Visible structure helps students follow the logic

- Part 1: Why OOP matters in backend systems.
- Part 2: Classes, objects, attributes, methods, and self.
- Part 3: Constructors, inheritance, and simple design patterns.
- Part 4: File handling, paths, errors, JSON, and small data storage.
- Part 5: Combining OOP and files in a mini backend-style example.



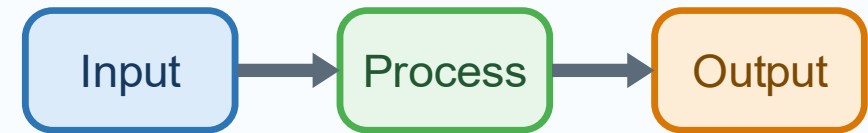
Teaching move

Keep pointing back to the roadmap so students can place each detail inside a bigger picture.

Before We Start

Bridge from Lecture 2 to Lecture 3

- Last lecture focused on variables, conditions, loops, functions, and simple files.
- Today we organize code into reusable units called classes.
- We also learn how programs can save data outside memory using files.
- Together, these skills help students move toward real applications.



How to say it

Functions organize actions; classes organize data and actions together. Files preserve data after the program closes.

Part 1 - Why OOP for Backend?

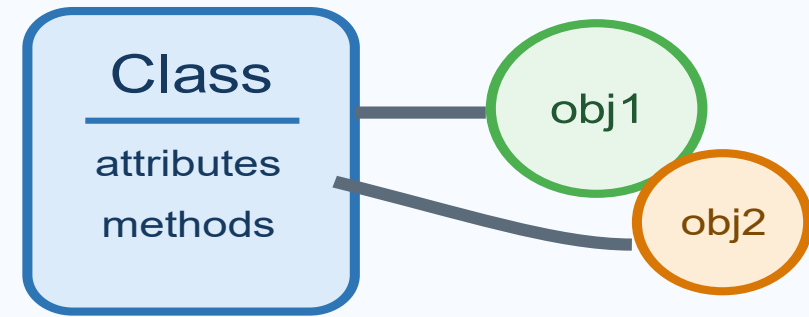
Students need the reason before the syntax.



What Problem Does OOP Solve?

Programs grow; structure becomes necessary

- A tiny script can survive with loose variables and functions.
- A larger application needs a clearer model of its real entities.
- Backend systems often deal with users, products, courses, orders, and messages.
- Object-oriented programming lets us represent such entities more naturally.



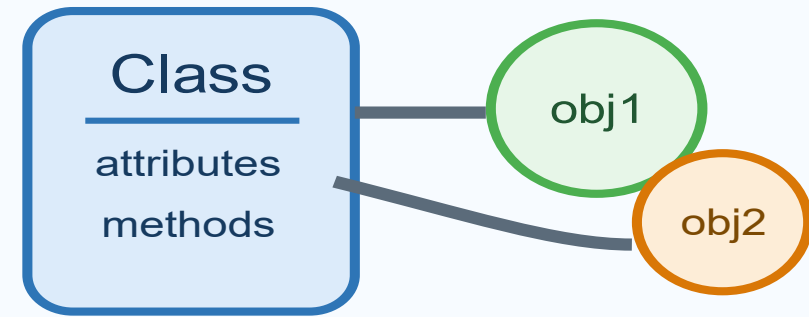
Real-world framing

Ask: if a system stores hundreds of students, should we keep random variables, or should we create a Student model?

OOP in One Sentence

The shortest definition that students can remember

- Object-oriented programming is a way to build programs using classes and objects.
- A class defines the blueprint.
- An object is a real instance created from that blueprint.
- The object holds data and can perform actions through methods.



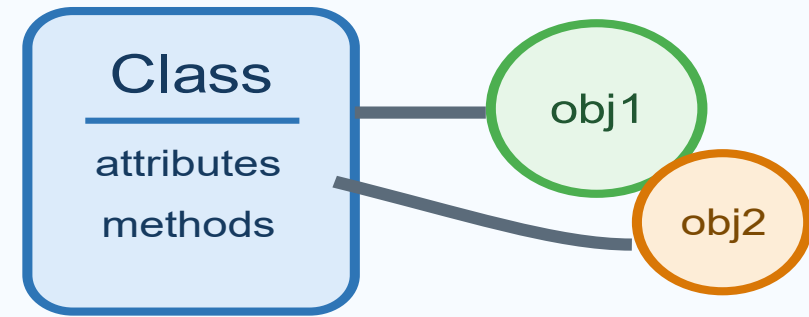
Instructor Note

Repeat “blueprint versus instance” several times. This is the key mental model.

Why Backend Developers Use OOP

Link the topic to future Django understanding

- Backend systems model real things: Student, Course, User, Product, Invoice.
- OOP groups related data and behavior in one place.
- This improves readability, reuse, testing, and maintenance.
- Django models themselves are classes, so this lecture prepares students for Django deeply.



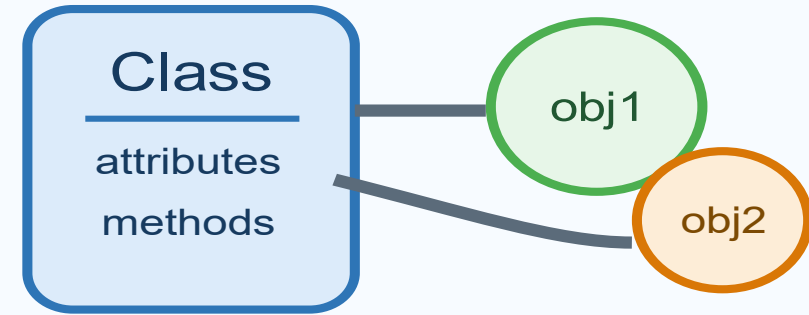
Strong connection

Say clearly: every time you define a Django model later, you are writing a class.

Real-World Analogy: A Student System

Use an example from their own university context

- Suppose we want to represent students in a program.
- Each student has data such as name, level, and department.
- Each student can also do actions such as display information or update grades.
- A Student class lets us keep that logic together.



Prompt for students

Ask them to name other campus entities that can become classes: course, instructor, classroom, lab, timetable.

Quick Discussion Task

Practice is where understanding becomes skill

- If you had to design a program for library management, what classes would you create?
- For each class, mention two attributes and one behavior.
- Let pairs of students discuss for 2 minutes, then share answers.
- Write the best answers on the board.



Expected Direction

The goal is not exact syntax yet. The goal is seeing how software models real-world entities.

Part 2 - Class, Object, Attribute, Method

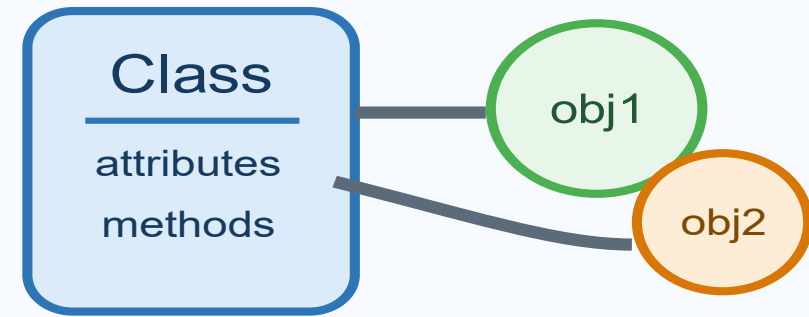
This section should move slowly and use repeated examples.



What Is a Class?

Start from the most basic concept

- A class is a template or blueprint.
- It tells Python what data an object should have and what actions it should support.
- A class does not represent one specific student or one specific course.
- It represents the general idea of such an entity.



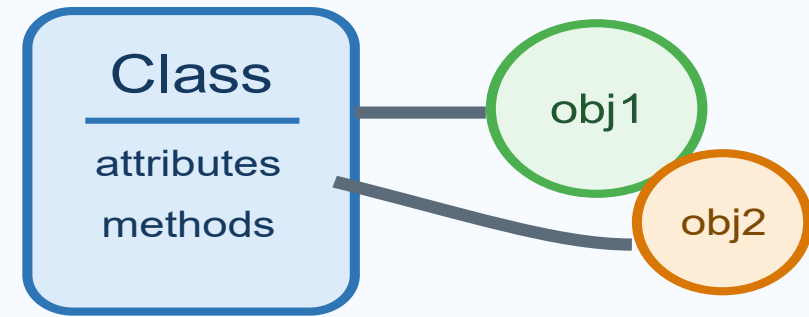
Explain carefully

The class itself is not “Mona” or “Ali”. It is the general plan used to create many objects.

What Is an Object?

Move from abstract idea to concrete instance

- An object is a real thing created from a class.
- If Student is the blueprint, then Mona and Ali can each be objects.
- Each object can have different values while still following the same class structure.
- This is why one class can produce many objects.



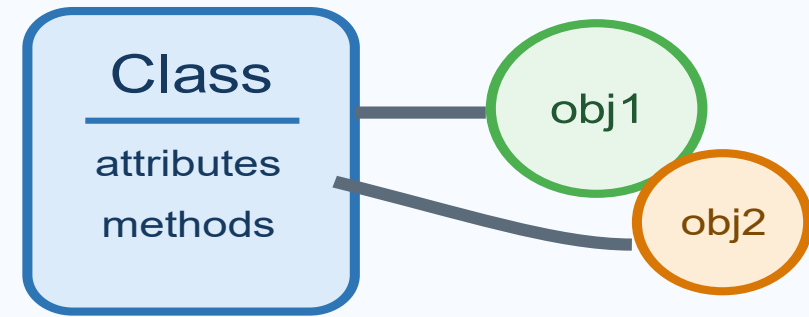
Board sketch

Draw one class box and then 2 or 3 objects below it with different values.

Class vs Object

This distinction must become automatic

- Class = design, plan, or definition.
- Object = actual instance created from that plan.
- The class describes what is possible.
- The object stores actual values and performs actual actions.



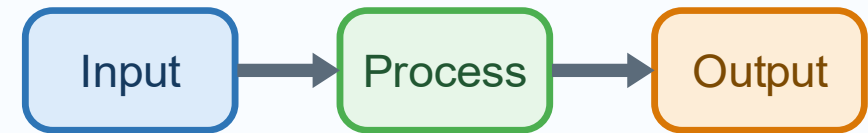
Simple phrase

Tell students: “Class is like a form; object is a filled form.”

Attributes and Methods

The two core ingredients inside classes

- Attributes are the data stored by an object.
- Methods are functions defined inside the class.
- Attributes answer: what does the object know?
- Methods answer: what can the object do?



Useful line

Say: data + behavior together = object-oriented thinking.

The Simplest Class

First syntax exposure should be tiny

- Use the class keyword followed by a class name.
- By convention, class names start with CapitalLetters.
- pass means the class is empty for now.
- This is only structure; no object exists yet.

Live Code

```
1 class Student:  
2     pass
```

Teaching Tip

Pause and explain indentation. Students should notice that Python uses blocks through indentation, not braces.

Creating Objects from a Class

Instantiation is the action of creating an object

- After defining the class, call the class name like a function.
- Each call creates a new object.
- Different variables can store different objects.
- These objects currently have no custom data yet.

Live Code

```
1 class Student:
2     pass
3
4 student1 = Student()
5 student2 = Student()
6
7 print(student1)
8 print(student2)
```

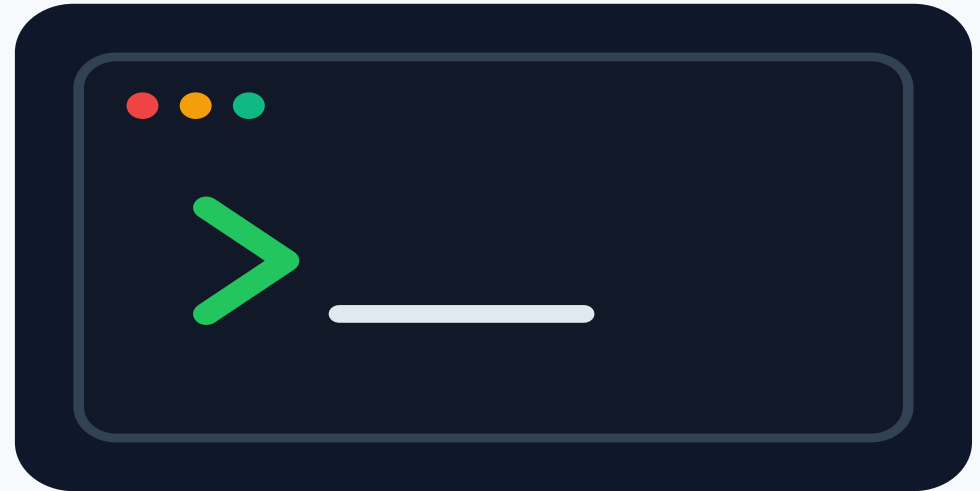
Teaching Tip

Run it to show that Python prints object addresses. This proves two distinct objects were created.

What Does print(student1) Show?

Students should understand what they are seeing

- Python prints a textual representation of the object.
- The exact memory-style address is not important for now.
- What matters is that student1 and student2 are separate objects.
- Later we can define better output using special methods, but not today.



Reassure students

Do not let them get stuck on the printed address. Focus on the idea of distinct instances.

Adding Attributes Manually

Show the first way to attach data to an object

- Python lets us assign attributes to an object after creating it.
- Each object can store different values.
- This works, but it becomes messy for larger programs.
- That is why constructors become important.

Live Code

```
1 class Student:
2     pass
3
4 student1 = Student()
5 student1.name = "Mona"
6 student1.level = 2
7
8 print(student1.name)
9 print(student1.level)
```

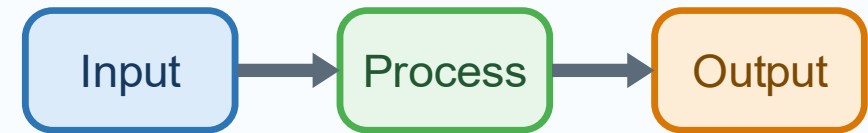
Teaching Tip

Use this slide to motivate why we need `__init__`: creating many objects this way is repetitive and risky.

Why Manual Attributes Are Weak

Teach motivation, not only syntax

- We may forget to create an important attribute.
- Different objects may end up with inconsistent structure.
- The code becomes repetitive when many objects are created.
- A constructor solves these problems by standardizing object creation.



Bridge slide

This is the “why” behind `__init__`. Students remember better when they first feel the problem.

The Constructor: `__init__`

The most important method for beginner OOP

- `__init__` runs automatically when an object is created.
- It receives the initial values needed to build the object.
- This makes object creation consistent and clean.
- Almost every beginner class starts here.

Live Code

```
1 class Student:
2     def __init__(self, name, level, department):
3         self.name = name
4         self.level = level
5         self.department = department
```

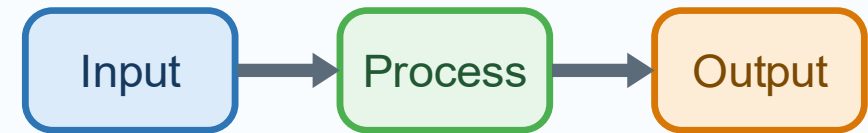
Teaching Tip

Underline the double underscore before and after `init`. Then ask: who calls this method? Python calls it automatically.

What Does self Mean?

This deserves extra care and repetition

- self refers to the current object being created or used.
- It allows the object to store and access its own attributes.
- self.name means “the name attribute of this specific object”.
- Without self, Python would not know which object the data belongs to.



Best explanation

Say: self is not a special keyword, but it is the standard and expected name for the current instance.

Creating Objects with `__init__`

Now object creation becomes cleaner

- Values are passed directly during object creation.
- The constructor stores them into self attributes.
- Each object still has its own data.
- The code is shorter and easier to read.

Live Code

```
1 class Student:
2     def __init__(self, name, level, department):
3         self.name = name
4         self.level = level
5         self.department = department
6
7 student1 = Student("Mona", 2, "IS")
8 student2 = Student("Ali", 2, "CS")
9
10 print(student1.name)
11 print(student2.department)
```

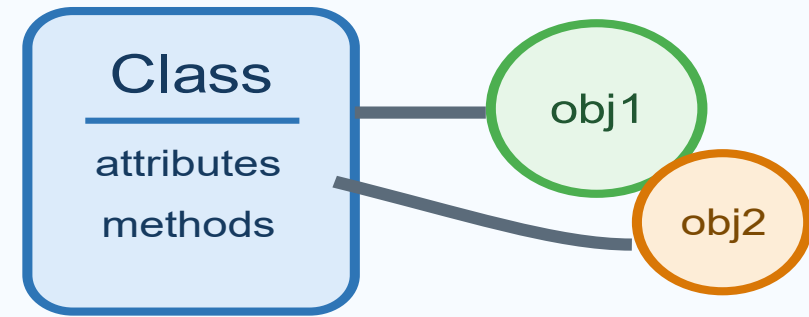
Teaching Tip

Run this and ask students to predict the output before pressing Enter.

Attributes Belong to Objects

A crucial concept for avoiding confusion

- `student1.name` and `student2.name` can hold different values.
- Both objects came from the same class, but they are independent instances.
- Changing one object does not automatically change another object.
- This is why objects are useful for representing many similar entities.



Live prompt

Ask: if `student1.name` changes to “Nour”, does `student2.name` also change? Why not?

Adding a Method

Now add behavior, not just data

- A method is a function defined inside a class.
- Methods also use `self` to access object attributes.
- This lets the object perform meaningful actions.
- Methods make objects feel active, not just passive data containers.

Live Code

```
1 class Student:
2     def __init__(self, name, level, department):
3         self.name = name
4         self.level = level
5         self.department = department
6
7     def display_info(self):
8         print(self.name, self.level, self.department)
9
10 student1 = Student("Mona", 2, "IS")
11 student1.display_info()
```

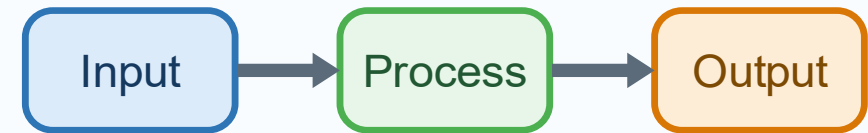
Teaching Tip

Pause at the dot notation: `student1.display_info()` means call the method on that object.

Method Call Flow

Break the invisible process into steps

- The object `student1` already exists.
- `student1.display_info()` calls the method `display_info`.
- Inside the method, `self` becomes `student1`.
- So `self.name` actually means `student1.name` during this call.



Very important

This is where many students finally understand `self`. Use object substitution explicitly.

Print vs Return Inside Methods

Students should not confuse console output with returned values

- print shows a result on the screen.
- return sends a value back to the caller.
- Methods can do either one depending on the need.
- Backend functions often return values for later use, not only display them.

Live Code

```
1 class Course:
2     def __init__(self, title, hours):
3         self.title = title
4         self.hours = hours
5
6     def description(self):
7         return f"{self.title} - {self.hours} hours"
8
9 course1 = Course("Python", 3)
10 print(course1.description())
```

Teaching Tip

Ask what would happen if description used print instead of return.

Class Task - Build a Simple Course Class

Practice is where understanding becomes skill

- Create a Course class with title, code, and hours.
- Add a method called show_course() that prints the data.
- Create two Course objects with different values.
- Run the method for both objects.



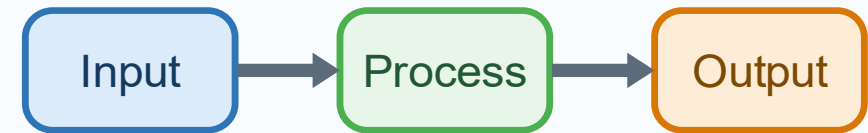
Expected Direction

Walk around and check two things: correct indentation and correct use of self.

Naming Conventions in OOP

Style supports readability

- Class names usually use PascalCase: Student, Course, LibraryBook.
- Attributes and method names usually use snake_case: student_name, display_info.
- Choose names that explain meaning, not short unclear names.
- Readable code is easier to debug and maintain.



Relate to Lecture 2

Connect this to good variable naming. OOP still depends on clear names.

Part 3 - Constructors, Inheritance, and Design

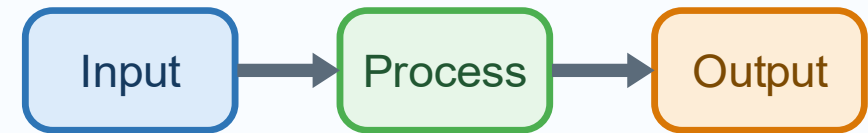
Go one level deeper than basic classes.



Why Constructors Matter So Much

A constructor creates discipline

- It guarantees that each object starts with the required attributes.
- It reduces repeated code during object creation.
- It makes the class easier to understand because initialization is explicit.
- It lowers the chance of inconsistent objects.



Use contrast

Compare a constructor-based class to the earlier manual-attribute example. Students should feel why this is better.

Constructor with a Default Value

This is a practical extension students often need

- Some attributes can have a standard default.
- If the caller does not provide a value, Python uses the default.
- This keeps the object flexible while staying organized.
- Defaults are common in real backend code.

Live Code

```
1 class Student:
2     def __init__(self, name, level,
3 department="General"):
4         self.name = name
5         self.level = level
6         self.department = department
7
8 student1 = Student("Mona", 2)
9 student2 = Student("Ali", 2, "CS")
10
11 print(student1.department)
12 print(student2.department)
```

Teaching Tip

This slide helps students see that constructors can be friendly, not rigid.

Updating Object Data Through Methods

Objects should manage their own state cleanly

- Methods can update attributes in a controlled way.
- This is better than changing values randomly all over the program.
- It keeps related logic inside the class.
- This is the start of cleaner software design.

Live Code

```
1 class Student:
2     def __init__(self, name, level):
3         self.name = name
4         self.level = level
5
6     def promote(self):
7         self.level += 1
8
9 student1 = Student("Mona", 2)
10 student1.promote()
11 print(student1.level)
```

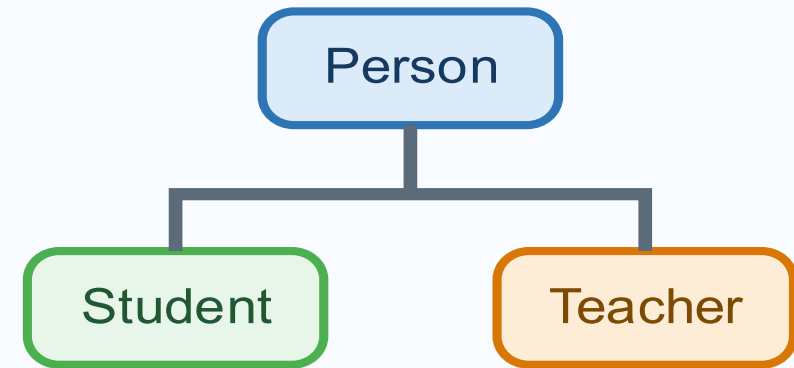
Teaching Tip

Ask students to explain why level changed. Make them speak the sequence of events.

What Is Inheritance?

Students should see it as “extension”, not magic

- Inheritance allows a new class to reuse data and methods from an existing class.
- The existing class is often called the parent or base class.
- The new class is called the child or derived class.
- This avoids repeating common logic.



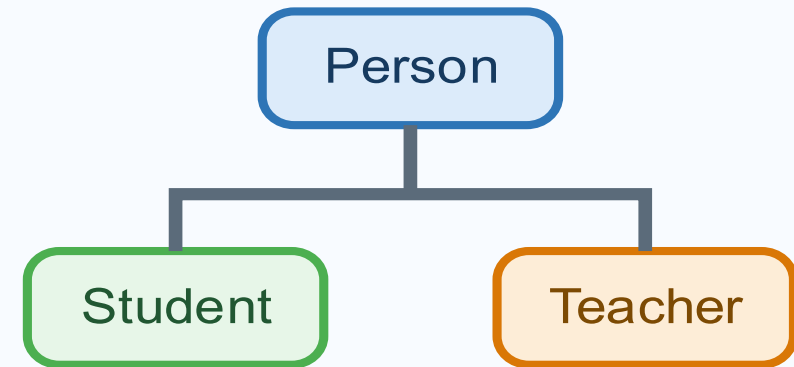
Simple phrase

Say: inheritance means “is a special kind of”. A Student is a kind of Person.

Why Inheritance Is Useful

Explain the benefit before showing syntax

- Many classes share common information such as name or ID.
- We do not want to rewrite the same code in many places.
- Inheritance collects shared logic in one parent class.
- Child classes can then add their own extra features.



Example set

Person -> Student, Teacher. Vehicle -> Car, Bus. Account -> AdminAccount, CustomerAccount.

A First Inheritance Example

Keep the first example simple and readable

- Person holds shared data.
- Student inherits from Person.
- Student adds one extra attribute called level.
- The child class still uses the parent constructor through super().

Live Code

```
1 class Person:
2     def __init__(self, name):
3         self.name = name
4
5 class Student(Person):
6     def __init__(self, name, level):
7         super().__init__(name)
8         self.level = level
9
10 student1 = Student("Mona", 2)
11 print(student1.name)
12 print(student1.level)
```

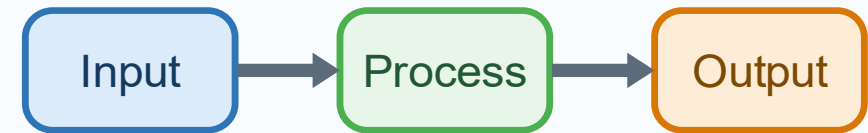
Teaching Tip

Explain super() as a way to call the parent class logic instead of rewriting it.

What Does super() Do?

A common beginner question

- `super()` gives access to methods from the parent class.
- In constructors, it is often used to initialize shared attributes.
- This avoids copying parent code into the child class.
- It keeps the relationship clean and maintainable.



Teaching note

You do not need deep theory here. Just teach the practical idea: reuse the parent initialization.

Method Overriding

Show how a child class can customize behavior

- A child class can redefine a method from the parent class.
- This is called overriding.
- The child keeps the same method name but changes the behavior.
- This is useful when classes share a concept but not identical output.

Live Code

```
1 class Person:
2     def introduce(self):
3         print("I am a person")
4
5 class Student(Person):
6     def introduce(self):
7         print("I am a student")
8
9 p = Person()
10 s = Student()
11 p.introduce()
12 s.introduce()
```

Teaching Tip

Run this and ask why two outputs differ even though the method name is the same.

Class Task - Design an Inheritance Example

Practice is where understanding becomes skill

- Create a base class called Animal with a name attribute.
- Create a child class called Cat that adds a color attribute.
- Print both the name and the color.
- If time allows, add a speak() method in both classes.



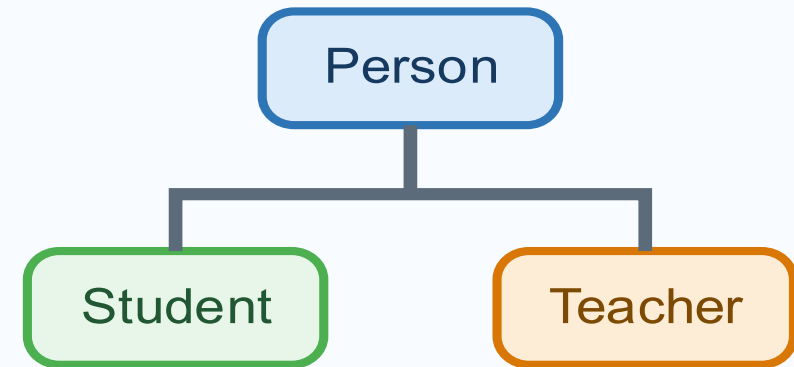
Expected Direction

The goal is to practice parent-child structure and `super()`, not to create a perfect project.

Inheritance vs Repetition

A design-thinking slide

- If two classes share important behavior, inheritance may help.
- If they only happen to look similar once, repetition may be simpler.
- Do not use inheritance just because it exists.
- Use it when the relationship is meaningful and improves clarity.



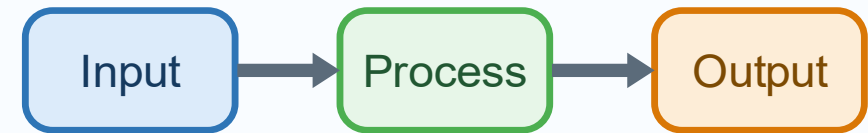
Instructor note

This slide builds judgment, not just syntax. Keep it simple and practical.

Public vs “Private” in Beginner Python

Introduce the idea gently

- Python does not enforce privacy like some other languages do.
- However, a leading underscore `_name` suggests internal use by convention.
- For beginners, the main lesson is to treat important data carefully.
- Later, larger frameworks rely on discipline and conventions a lot.



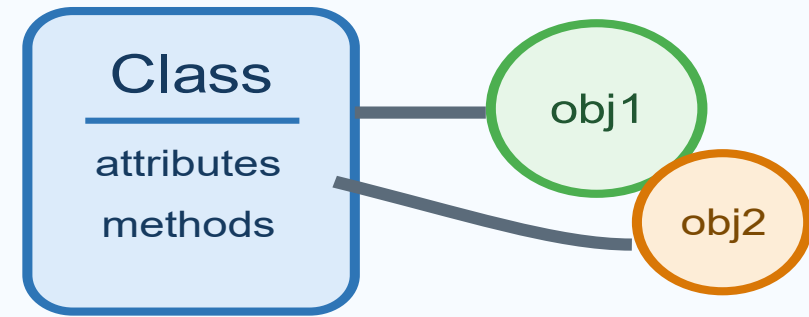
Keep it light

Do not turn this into advanced theory. Just plant the idea that naming can signal design intention.

Composition: Another Design Idea

Not everything should be inheritance

- Composition means one object contains another object as part of its structure.
- A Course may have an Instructor object, not inherit from Instructor.
- A Car has an Engine; it is not a type of Engine.
- This helps students think more carefully about relationships.



Useful sentence

Inheritance is “is a”; composition is “has a”.

Part 4 - File Handling in Python

Now shift from memory to permanent data storage.



Why Do We Need Files?

Programs often need data after they stop running

- Variables disappear when the program ends.
- Files let us store data for later use.
- Backend systems save logs, reports, settings, uploads, and exported data.
- File handling is the first step toward thinking about persistence.



Backend link

Before databases, help students understand the simpler idea of storing information outside RAM.

Common Backend Uses of Files

Make the topic feel real

- Saving text reports and logs.
- Reading configuration values or templates.
- Exporting data to CSV or text files.
- Working with JSON files for structured data exchange.



Prompt

Ask students which of these they have already seen in websites or apps, even if they did not know the implementation.

The open() Function

Introduce the doorway to file work

- `open()` is used to access a file.
- It usually needs a file name and a mode.
- The mode tells Python what we want to do: read, write, append, and so on.
- After using the file, we should close it or use with `open(...)`.



Core habit

Teach “open, use, close” as a simple lifecycle. Then improve it with `with open`.

File Modes You Must Know

Students should memorize these early

- r = read from an existing file.
- w = write to a file; creates or overwrites content.
- a = append to the end of a file.
- x = create a new file and fail if it already exists.



Warning

Emphasize that w can erase old content. Many beginners lose data here.

Writing to a Text File

The first complete file example

- Open the file in write mode.
- Use write() to send text to the file.
- Close the file after writing.
- Then open the created file to inspect the result.

Live Code

```
1 file = open("students.txt", "w")
2 file.write("Mona
3 ")
4 file.write("Ali
5 ")
6 file.close()
7
8 print("File written successfully")
```

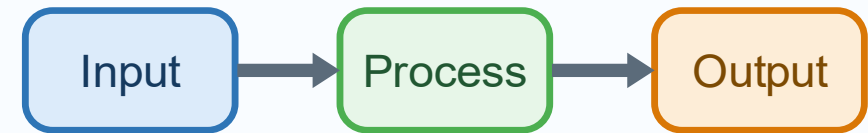
Teaching Tip

Run it, then open the file visually in VS Code so students see the real saved text.

What Does Mean?

A tiny but necessary explanation

- is the newline character.
- It moves the next text to a new line.
- Without it, written content may appear on one long line.
- Text formatting matters when people read output files.



Micro-demo

Remove
once and run again so students feel the difference.

Reading from a File

The reverse of writing

- Open the file in read mode.
- Use `read()` to get the full content as one string.
- Store the content in a variable.
- Print the result to the terminal.

Live Code

```
1 file = open("students.txt", "r")
2 content = file.read()
3 file.close()
4
5 print(content)
```

Teaching Tip

Explain that content now lives in memory as a Python string.

Appending New Data

Keep old content and add more

- Append mode adds text to the end of the existing file.
- It is useful for logs, records, and gradual updates.
- Unlike w mode, it does not delete the old content.
- This difference is essential in backend tasks.

Live Code

```
1 file = open("students.txt", "a")
2 file.write("Nour
3 ")
4 file.close()
5
6 print("New line added")
```

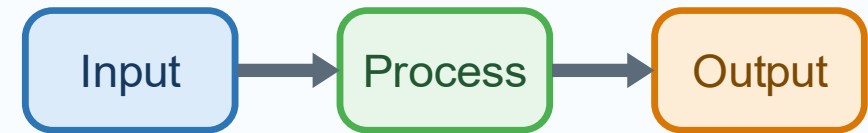
Teaching Tip

Ask students to predict the final file content before opening the file.

read(), readline(), and readlines()

Different ways to read text

- read() returns the whole file as one string.
- readline() returns only one line at a time.
- readlines() returns a list of lines.
- The best choice depends on the task and file size.



Teaching idea

This is a good moment to connect file reading to lists from Lecture 2.

Using readlines()

Show how file data becomes a list

- Each line becomes one element in the list.
- We can loop over the lines easily.
- This is useful when processing records one by one.
- Remember that newline characters may still be present.

Live Code

```
1 file = open("students.txt", "r")
2 lines = file.readlines()
3 file.close()
4
5 for line in lines:
6     print(line.strip())
```

Teaching Tip

Explain strip() as removing extra whitespace and newline characters.

The with Statement

Teach the safer modern style

- with open(...) automatically closes the file when the block ends.
- This is safer and cleaner than remembering close() manually.
- It reduces the chance of leaving files open by mistake.
- In professional Python code, this is the preferred pattern.



Strong recommendation

After this slide, prefer with open in almost all later examples.

Writing with with open

Students should start adopting the better style immediately

- Open the file using with open(...).
- Write inside the indented block.
- No explicit close() is needed.
- When the block ends, Python closes the file automatically.

Live Code

```
1 with open("courses.txt", "w") as file:
2     file.write("Backend Python
3 ")
4     file.write("Django
5 ")
6
7 print("Done")
```

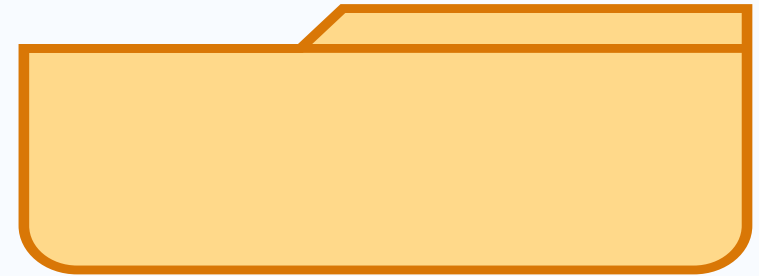
Teaching Tip

This is a slide worth repeating verbally: safer, shorter, cleaner.

Relative Paths and Working Folder

Many file errors are really path errors

- A relative path is based on the current working folder.
- If the file is not where Python expects, `open()` will fail.
- Students should know where their script is running from.
- Good folder organization prevents confusion.



Live demonstration

Show the location of the `.py` file and the created text file inside the same folder.

Checking the Current Folder

Useful for debugging file path problems

- Python can tell us the current working directory.
- This helps explain why a file is or is not found.
- It is a practical debugging habit.
- Do not memorize blindly; understand why it helps.

Live Code

```
1 import os
2
3 print(os.getcwd())
```

Teaching Tip

Run this in class and compare the displayed path to the folder students opened in VS Code.

Common File Errors

Prepare students for realistic problems

- `FileNotFoundError` appears when a file path is wrong or the file does not exist.
- `PermissionError` may appear when the program cannot access the file.
- Using the wrong mode may overwrite data or fail to read content.
- Most file bugs are solved by checking path, mode, and spelling.



Error mindset

Teach students not to panic. Read the error message carefully before changing random code.

Handling File Errors with try/except

Beginner-friendly error handling for files

- try runs the risky code.
- except catches an expected error and handles it gracefully.
- This is better than letting the program crash without explanation.
- Backend systems often use this pattern around input and file operations.

Live Code

```
1 try:
2     with open("missing.txt", "r") as file:
3         print(file.read())
4 except FileNotFoundError:
5     print("The file does not exist.")
```

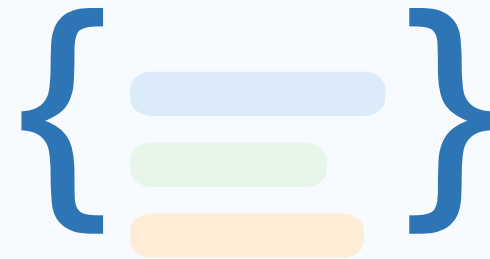
Teaching Tip

This slide also reinforces the error handling ideas from Lecture 2.

CSV and JSON: Two Common Data Formats

Move from plain text to structured files

- CSV is useful for table-like data such as rows and columns.
- JSON is useful for structured key-value data.
- Backend systems often exchange JSON with frontends and APIs.
- So JSON is especially important for web-related work.



Future connection

Tell students that Django views and APIs often work with JSON-like structures.

A Simple JSON Example

Keep JSON very concrete

- Python dictionaries map naturally to JSON-like thinking.
- Use the json module to save structured data.
- This is more expressive than raw text for many tasks.
- It prepares students for API-style data later.

Live Code

```
1 import json
2
3 student = {
4     "name": "Mona",
5     "level": 2,
6     "department": "IS"
7 }
8
9 with open("student.json", "w") as file:
10     json.dump(student, file, indent=4)
```

Teaching Tip

After running, open the JSON file so students see the readable structure.

Reading JSON Back into Python

Show the round-trip idea

- Open the JSON file in read mode.
- Use `json.load()` to convert file content back into a Python object.
- The result is often a dictionary or list.
- Now the data can be used by the program again.

Live Code

```
1 import json
2
3 with open("student.json", "r") as file:
4     data = json.load(file)
5
6 print(data["name"])
7 print(data["department"])
```

Teaching Tip

This is a beautiful moment to connect file handling, dictionaries, and future APIs.

Class Task - Choose the Correct File Mode

Practice is where understanding becomes skill

- If you want to read an existing file, which mode do you use?
- If you want to add a line without deleting old content, which mode do you use?
- If you want to create a fresh report and replace any old version, which mode do you use?
- Discuss answers in pairs before sharing.



Expected Direction

Expected answers: r for read, a for append, w for overwrite or fresh writing.

Part 5 - Combining OOP and Files

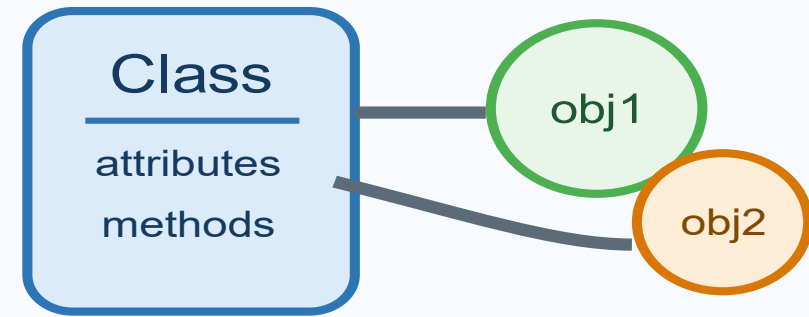
Now integrate both topics into one coherent example.



Why Combine OOP and Files?

This looks more like real software

- Objects help us organize program logic.
- Files help us preserve data between runs.
- Together they allow us to build small systems instead of isolated scripts.
- This is a strong preparation step before databases and Django models.



Big picture

Tell students: a backend app often models entities as objects and stores or exchanges their data somehow.

Mini Example: Student Object to Dictionary

Objects are not written directly to JSON without conversion

- JSON understands basic data structures like dict and list.
- A custom object should be converted into a dictionary first.
- This is an important design step.
- It also teaches students how to think about serialization.

Live Code

```
1 class Student:
2     def __init__(self, name, level, department):
3         self.name = name
4         self.level = level
5         self.department = department
6
7     def to_dict(self):
8         return {
9             "name": self.name,
10            "level": self.level,
11            "department": self.department
12        }
```

Teaching Tip

Introduce the word “serialize” briefly: turn object data into a savable format.

Saving a List of Objects to JSON

A practical combined example

- Create several Student objects.
- Convert each object into a dictionary.
- Store the dictionaries in a list.
- Write that list into a JSON file.

Live Code

```
1 import json
2
3 students = [
4     Student("Mona", 2, "IS"),
5     Student("Ali", 2, "CS")
6 ]
7
8 data = [student.to_dict() for student in students]
9
10 with open("students.json", "w") as file:
11     json.dump(data, file, indent=4)
```

Teaching Tip

Walk slowly through the list comprehension. This is a rich slide with many ideas combined.

Reading JSON and Recreating Objects

Reverse the process clearly

- Load the JSON file into a Python list of dictionaries.
- Loop through the dictionaries.
- Create Student objects again from the stored values.
- Now the program regains object-based structure from file data.

Live Code

```
1 import json
2
3 with open("students.json", "r") as file:
4     data = json.load(file)
5
6 students = []
7 for item in data:
8     student = Student(item["name"], item["level"],
9         item["department"])
9     students.append(student)
```

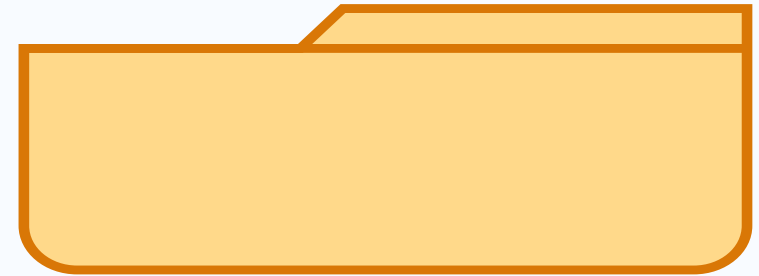
Teaching Tip

This slide shows how persistence and OOP cooperate.

A Simple Project Structure

Students need folder habits, not only code

- project_folder/
 - main.py
 - models.py
 - students.json
- Separating files by role improves readability and reuse.



Good habit

Even in small class examples, show structure. It will make Django feel less intimidating later.

Running a Multi-File Project

Students should know the execution flow

- Put the Student class inside models.py.
- Import Student into main.py.
- Run main.py from the terminal.
- Keep all files inside the same project folder.

Live Code

```
1 # models.py
2 class Student:
3     def __init__(self, name, level):
4         self.name = name
5         self.level = level
6
7 # main.py
8 from models import Student
9
10 student1 = Student("Mona", 2)
11 print(student1.name)
```

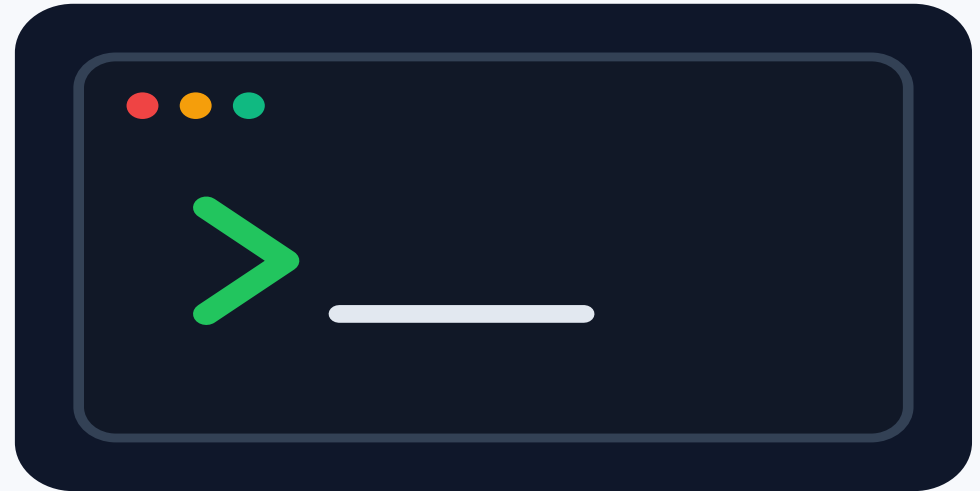
Teaching Tip

Use this to show that larger programs are made of multiple files, not one giant script.

Debugging Tips for Today's Lecture

Explicitly teach the debugging mindset

- Check indentation first when Python gives a syntax-related error.
- Check spelling of class names, method names, and file names.
- Check whether self is missing in methods.
- Check file mode and path when file operations fail.



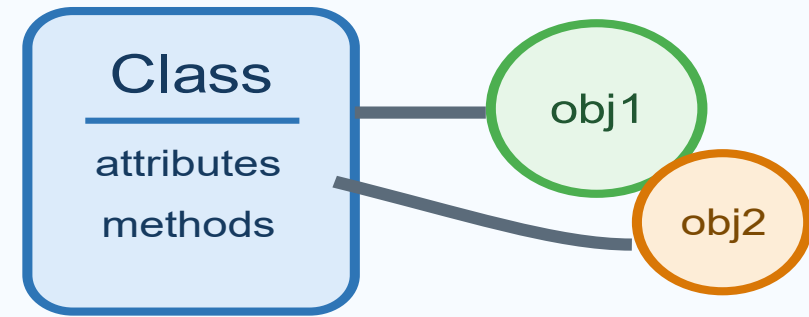
Calm students down

Tell them most beginner errors are small and local.
Debugging is reading carefully, not guessing wildly.

Common Beginner Mistakes in OOP

Make the invisible pain visible

- Forgetting self in a method definition.
- Using a constructor parameter but forgetting to save it with self.
- Trying to use an attribute before it exists.
- Calling a method without parentheses when trying to execute it.



Make it interactive

Ask students which one they already made during the lecture. Normalize mistakes.

Common Beginner Mistakes in File Handling

Prevent frustration before homework

- Opening the wrong file name or wrong folder.
- Using `w` when `a` was intended.
- Forgetting that `read()` returns a string, not a list.
- Writing JSON-like text manually instead of using `json.dump()`.



Teacher move

Give one concrete example of each mistake. Students learn well from near-errors.

Mini Quiz 1

Use these questions to check understanding before moving on

- What is the difference between a class and an object?
- Why do we use self inside methods?
- What is the role of `__init__`?
- What is one advantage of inheritance?

How to use

Read each question, pause, collect answers orally, then ask students to justify why their answer is correct.

Mini Quiz 2

Use these questions to check understanding before moving on

- What is the difference between w mode and a mode?
- Why is with open(...) recommended?
- What Python data structure is commonly loaded from a JSON object?
- Why might FileNotFoundError happen?

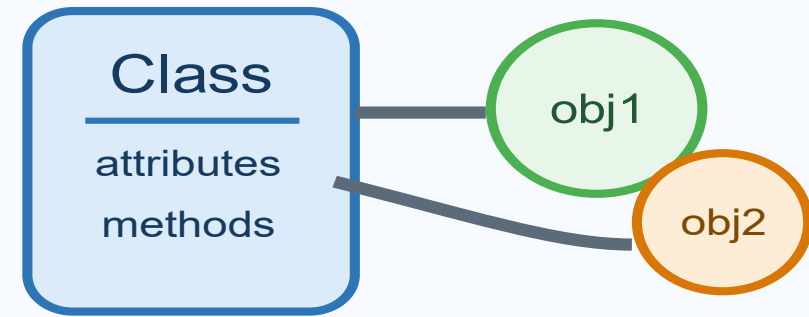
How to use

Read each question, pause, collect answers orally, then ask students to justify why their answer is correct.

Lecture Recap - OOP

Compress the first half into memorable ideas

- A class is a blueprint; an object is an instance.
- Attributes store data; methods define behavior.
- `self` refers to the current object.
- Constructors initialize objects, and inheritance supports reuse.



Recap style

Ask students to repeat each pair aloud: class/object, attribute/method, parent/child.

Lecture Recap - File Handling

Compress the second half into practical rules

- Files store data beyond program execution.
- The main modes are r, w, and a.
- with open(...) is safer and cleaner.
- JSON is a practical structured format for backend-style data.



Recap style

Keep this short and confident. Students should leave with a small clear list to remember.

Homework

Practice is where understanding becomes skill

- Create a Book class with title, author, and pages.
- Add a method that returns a short description string.
- Create at least three Book objects and save them to a JSON file.
- Read the file again and print the loaded data.



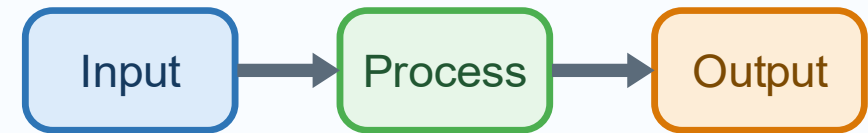
Expected Direction

This homework checks class design, object creation, method use, JSON writing, and JSON reading in one task.

Preview of Lecture 4

Keep the course connected

- The next lecture moves to web basics before Django.
- We will explain HTTP, request, response, URL, domain, and status code.
- This means we now move from “Python programs” to “web communication”.
- Everything from Lectures 1 to 3 will support that transition.



Close with anticipation

Students should feel that each lecture is building toward real web applications.

End of Lecture 3

Questions, live code review, or extra practice if time remains

- Re-run one class example slowly if students still feel uncertain.
- Re-run one file example and let a student explain each line.
- Invite students to ask about `self`, `__init__`, and file modes one more time.
- Encourage them to practice by typing, not only reading.



Final teaching tip

The lecture succeeds if students can explain the logic in their own words, not only copy syntax.